

CRYPTOLINE

May 11, 2026

1 Introduction

CRYPTOLINE is a tool and a language for the verification of low-level implementations of mathematical constructs. In CRYPTOLINE, users can specify two kinds of properties, namely algebraic properties and range properties. Algebraic properties involve equalities and modular equalities in the integer domain while range properties involve bit-accurate variable ranges. CRYPTOLINE verifies algebraic properties and range properties separately. Verification of algebraic properties is reduced to ideal membership queries which are solved by external computer algebra systems. Verification of range properties is reduced to Satisfiability Modulo Theories (SMT) queries which are solved by external SMT solvers.

2 CryptoLine Language

An *identifier* is a regular string started by a letter or an underscore, followed by letters, digits, or underscores.

$$id ::= (letter \mid underscore)[letter \mid digit \mid underscore]$$

All constants and variables in CRYPTOLINE are typed. Let w be a positive integer. $uintw$ and $sintw$ in CRYPTOLINE denote the types of bit-vectors with width w in the unsigned and two's complement signed representations respectively. The type $uint1$ is also written as bit .

$$typ ::= uint1 \mid sint2 \mid uint2 \mid sint3 \mid \dots \mid uintw \mid sint(w + 1)$$

A *constant* is an integer, a binary number, a hexadecimal number, a named

constant, or arithmetic expressions over constants.

$$\begin{array}{lcl}
const & ::= & simple_const \\
& | & (complex_const) \\
simple_const & ::= & \mathbb{Z} \\
& | & 0b[0 - 1]^+ \\
& | & 0x[0 - 9a - fA - F]^+ \\
& | & \$id \\
complex_const & ::= & const \\
& | & - complex_const \\
& | & complex_const + complex_const \\
& | & complex_const - complex_const \\
& | & complex_const * complex_const \\
& | & complex_const ** complex_const \\
typed_const & ::= & const@typ \\
& | & typ const
\end{array}$$

The value of a named integer c is read by $\$c$. CRYPTO_{LINE} supports the following arithmetic operators over constants: unary minus (-), addition (+), subtraction (-), multiplication (*), and exponent (**). A *typed constant* is a constant with its type explicitly specified.

A *variable* is an identity. A *typed variable* is a variable with its type explicitly specified. An *lval* is either a variable or a typed variable.

$$\begin{array}{lcl}
var & ::= & id \\
typed_var & ::= & var@typ \mid typ\ var \\
lval & ::= & var \mid typed_var
\end{array}$$

The notation t_0^* and t_0^+ respectively represents a possibly empty and a non-empty sequence of $\circ$ -separated t .

An *atom* is either a typed constant, a variable, or a typed variable. It is not necessary to specify the variable type explicitly in an atom because CRYPTO_{LINE} can infer the type automatically.

$$atom ::= typed_const \mid var \mid typed_var$$

An *algebraic expression* is evaluated over $\mathbb{Z}$.

$$\begin{array}{lcl}
eexp & ::= & simple_const \mid var \\
& | & - eexp \mid eexp + eexp \\
& | & eexp - eexp \mid eexp * eexp \\
& | & eexp ** eexp \mid \mathbf{limbs}\ const\ [eexp^+] \\
& | & (eexp)
\end{array}$$

$\mathbf{limbs}\ n\ [e_1, \dots, e_m]$ represents $e_1 + e_2 2^n + e_3 2^{2n} + \dots + e_m 2^{(m-1)n}$. A *range expression* is evaluated over bit vectors. $const\ w\ n$ is a bit-vector of width w and value n . $\sim$ (*neg*) is logical negation. $!$ (*not*), $\&$ (*and*), $|$ (*or*), $\wedge$ (*xor*) are respectively bit-wise negation, bit-wise AND, bit-wise OR, and bit-wise XOR.

umod is unsigned remainder. *srem* is 2's complement signed remainder (sign follows dividend). *smod* is 2's complement signed remainder (sign follows divisor). *uext* and *sxt* are respectively unsigned and signed extension operations.

<i>rexp</i> ::=	(<i>rexp</i>)		const <i>const const</i>
	− <i>rexp</i>		<i>rexp</i> + <i>rexp</i>
	<i>rexp</i> − <i>rexp</i>		<i>rexp</i> * <i>rexp</i>
	~ <i>rexp</i>		neg <i>rexp</i>
	! <i>rexp</i>		not <i>rexp</i>
	<i>rexp</i> & <i>rexp</i>		and <i>rexp rexp</i>
	<i>rexp</i> <i>rexp</i>		or <i>rexp rexp</i>
	<i>rexp</i> ^ <i>rexp</i>		xor <i>rexp rexp</i>
	umod <i>rexp rexp</i>		srem <i>rexp rexp</i>
	smod <i>rexp rexp</i>		limbs <i>const [rexp⁺]</i>
	uext <i>rexp const</i>		sxt <i>rexp const</i>

A *predicate* is represented by an algebraic predicate and a range predicate.

$$pred ::= \mathbf{true} \mid epred \ \&\& \ rpred$$

An *algebraic predicate* is evaluated over the integer domain. $e_1 = e_2$ (*eq* $e_1 \ e_2$) is an equality over algebraic expressions. $e_1 = e_2 \ (\text{mod} \ [m_1, \dots, m_n])$ (*eqmod* $e_1 \ e_2 \ [m_1, \dots, m_n]$) is a modular equality. $p_1 \ /\wedge \ p_2$ (*and* $p_1 \ p_2$) is a logical conjunction of p_1 and p_2 . The conjunction of a sequence of algebraic predicates $e_1, \dots, e_n$ is written as $\bigwedge [e_1, \dots, e_n]$ (*and* $[e_1, \dots, e_n]$).

<i>epred</i> ::=	(<i>epred</i>)		true
	<i>eexp</i> = <i>eexp</i>		eq <i>eexp eexp</i>
	<i>eexp</i> = <i>eexp</i> (mod $[eexp^+]$)		eqmod <i>eexp eexp</i> $[eexp^+]$
	<i>epred</i> /\ <i>epred</i>		and <i>epred epred</i>
	$\bigwedge [eexp^+]$		and $[eexp^+]$

A *range predicate* specifies the ranges of variables. CRYPTOLINE offers comparisons such as equality (=), modular equalities (*equmod*, *eqsmod*, *eqsrem*), unsigned less than (<), unsigned less than or equal to (<=), unsigned greater than (>), unsigned greater than or equal to (>=), signed less than (< *s*), signed less than or equal to (<= *s*), signed greater than (> *s*), and signed greater than

or equal to ($\geq s$).

<i>rpred</i>	::=	(<i>rpred</i>)		true
		<i>rexp</i> = <i>rexp</i>		eq <i>rexp</i> <i>rexp</i>
		<i>rexp</i> = <i>rexp</i> (<i>umod</i> <i>rexp</i>)		equmod <i>rexp</i> <i>rexp</i> <i>rexp</i>
		<i>rexp</i> = <i>rexp</i> (<i>smod</i> <i>rexp</i>)		eqsmod <i>rexp</i> <i>rexp</i> <i>rexp</i>
		<i>rexp</i> = <i>rexp</i> (<i>srem</i> <i>rexp</i>)		eqsrem <i>rexp</i> <i>rexp</i> <i>rexp</i>
		<i>rexp</i> < <i>rexp</i>		ult <i>rexp</i> <i>rexp</i>
		<i>rexp</i> <= <i>rexp</i>		ule <i>rexp</i> <i>rexp</i>
		<i>rexp</i> > <i>rexp</i>		ugt <i>rexp</i> <i>rexp</i>
		<i>rexp</i> >= <i>rexp</i>		uge <i>rexp</i> <i>rexp</i>
		<i>rexp</i> < s <i>rexp</i>		slt <i>rexp</i> <i>rexp</i>
		<i>rexp</i> <= s <i>rexp</i>		sle <i>rexp</i> <i>rexp</i>
		<i>rexp</i> > s <i>rexp</i>		sgt <i>rexp</i> <i>rexp</i>
		<i>rexp</i> >= s <i>rexp</i>		sge <i>rexp</i> <i>rexp</i>
		$\sim$ <i>rpred</i>		neg <i>rpred</i>
		<i>rpred</i> $\wedge$ <i>rpred</i>		and <i>rpred</i> <i>rpred</i>
		<i>rpred</i> $\vee$ <i>rpred</i>		or <i>rpred</i> <i>rpred</i>
		$\wedge$ [<i>rpred</i> ⁺]		and [<i>rpred</i> ⁺]
		$\vee$ [<i>rpred</i> ⁺]		or [<i>rpred</i> ⁺]

There are numerous *instructions* supported by CRYPTO LINE.

- *mov x a* assigns destination variable *x* by the value of the source atom *a*.
- *cmov x c a₁ a₂* assigns destination variable *x* by the value of the source atom *a₁* if the condition bit *c* is 1, and otherwise by the value of the source atom *a₂*.
- *add x a₁ a₂* assigns *x* by the addition of the source atoms *a₁* and *a₂*. Note that *add* may overflow.
- *adds c x a₁ a₂* assigns *x* by the addition of the source atoms *a₁* and *a₂* with carry bit *c* set.
- *adc x a₁ a₂ y* assigns *x* by the addition of the carry bit *y* and the source atoms *a₁* and *a₂*.
- *adcs* is the same as *adc* except the carry bit is set.
- There are also instructions *sub* for subtraction; *subs*, *subc*, *sbc* and *sbc*s for subtraction with carry; *subb*, *sbb*, and *sbb*s for subtraction with borrow.
- *mul* and *muls* are half multiplication operations. The difference is that *muls* sets the carry bit if the multiplication under- or over-flow.
- *mull* is full multiplication with results split into high part and low part.
- *mulj* is also full multiplication without splitting the results. *smulj* is the signed variant of *mulj*.

- *nondet* assigns a variable by a nondeterministic value.
- *set x* assigns the bit variable *x* by 1 while *clear x* assigns the bit variable *x* by 0.
- *shl x a n* shifts the source atom *a* left by *n* and stores the results in *x*.
- *shls o x a n* is the same as *shls x a n* except that the bits shifted out are stored in *o*.
- *shr x a n* shifts the source atom *a* right logically by *n* and stores the results in *x*.
- *shrs x o a n* is the same as *shr x a n* except that the bits shifted out are stored in *o*.
- *sar* and *sars* are the same as *shr* and *shrs* respectively except that the right shift is arithmetic.
- *cshl x_h x_l a₁ a₂ n* concatenates the source atoms *a₁* (high bits) and *a₂* (low bits), shifts the concatenation left by *n*, stores the high bits of the shifted concatenation in *x_h*, and stores the low bits shifted right by *n* in *x_l*.
- *cshls x_o x_h x_l a₁ a₂ n* is the same as *cshl x_h x_l a₁ a₂ n* except that the bits shifted out are stored in *x_o*.
- *cshr x_h x_l a₁ a₂ n* concatenates the source atoms *a₁* (high bits) and *a₂* (low bits), shifts the concatenation right logically by *n*, stores the high bits of the shifted concatenation in *x_h*, and stores the low bits in *x_l*.
- *cshrs x_h x_l x_o a₁ a₂ n* is the same as *cshr x_h x_l a₁ a₂ n* except that the bits shifted out are stored in *x_o*.
- *spl x_h x_l a n* splits the source atom *a* at position *n*, stores the high bits in *x_h*, and stores the low bits in *x_l*. *sspl* is the signed variant of *spl*. It splits the signed source atom, stores the signed high bits in *x_h* and the unsigned low bits in *x_l*.
- *split* is the same as *spl* except that the high bits and the low bits are extended to the size of *a*. *ssplit* is the signed variant of *split*. While the low bits are always zero extended, the high bits are sign extended if *a* is signed and otherwise zero extended.
- *join x a₁ a₂* assigns *x* by the concatenation of the source atoms *a₁* (high bits) and *a₂* (low bits).
- *seteq r a₁ a₂* sets *r* to all 1's if *a₁* equals *a₂*, and to all 0's otherwise. The type of the destination variable is bit by default if its type is not specified explicitly.

- *setne r a₁ a₂* sets *r* to all 1's if *a₁* does not equal *a₂*, and to all 0's otherwise. The type of the destination variable is **bit** by default if its type is not specified explicitly.
- *and*, *or*, *not*, and *xor* are bit-wise operations.
- *cast t x a* assigns *x* by the source atom *a* casted to the type *t*.
- *vpc t x a* is the same as *cast t x a* except that the integer interpretation of *x* must be the same as the integer interpretation of *a*.
- *assert* tells CRYPTO LINE to verify the specified predicate.
- *assume* tells CRYPTO LINE to assume the specified predicate.
- *cut e && r* is an alias of one *ecut e* followed by a *rcut r*. For *ecut*, CRYPTO LINE verifies the specified algebraic predicate and starts afresh with the predicate assumed when verifying algebraic properties. Similarly for *rcut*, CRYPTO LINE verifies the specified range predicate and starts afresh with the predicate assumed when verifying range properties.
- *ghost* can introduce logical variables that must only be used in specifications such as *assert*, *assume*, *cut*, *ecut*, *rcut*, and postconditions. The predicate in a *ghost* instruction is always assumed.
- *call p(a₁, a₂, ..., a_n)* invokes a defined procedure *p* with the actual parameters *a₁, a₂, ..., a_n* (call be value for input parameters). The call is replaced by the pre-/post-conditions of *p* for verification. Thus *call* is currently incompatible with equivalence checking and simulation.
- *inline p(a₁, a₂, ..., a_n)* inlines a defined procedure *p* with the actual parameters *a₁, a₂, ..., a_n*, which replace every occurrence of the corresponding formal parameters in the procedural body.
- *inlinespec p(a₁, a₂, ..., a_n)* is semantically the same as *inline* but *inlinespec* is replaced by the pre-/post-conditions of *p* for verification.

<i>instr</i>	::=	mov <i>lval atom</i>		cmov <i>lval lval atom atom</i>
		add <i>lval atom atom</i>		adds <i>lval lval atom atom</i>
		adc <i>lval atom atom var</i>		adcs <i>lval lval atom atom var</i>
		sub <i>lval atom atom</i>		subs <i>lval lval atom atom</i>
		subc <i>lval lval atom atom</i>		subb <i>lval lval atom atom</i>
		sbc <i>lval atom atom var</i>		sbc <i>lval lval atom atom var</i>
		sbb <i>lval atom atom var</i>		sbbs <i>lval lval atom atom var</i>
		mul <i>lval atom atom</i>		muls <i>lval lval atom atom</i>
		mull <i>lval lval atom atom</i>		mulj <i>lval atom atom</i>
		smulj <i>lval atom atom</i>		nondet <i>lval</i>
		set <i>lval</i>		clear <i>lval</i>
		shl <i>lval atom const</i>		shls <i>lval lval atom const</i>
		shr <i>lval atom const</i>		shrs <i>lval lval atom const</i>
		sar <i>lval atom const</i>		sars <i>lval lval atom const</i>
		cshl <i>lval lval atom atom const</i>		cshls <i>lval lval lval atom atom const</i>
		cshr <i>lval lval atom atom const</i>		cshrs <i>lval lval lval atom atom const</i>
		spl <i>lval lval atom const</i>		sspl <i>lval lval atom const</i>
		split <i>lval lval atom const</i>		ssplit <i>lval lval atom const</i>
		join <i>lval lval atom const</i>		
		seteq <i>lval atom atom</i>		setne <i>lval atom atom</i>
		and <i>lval atom atom</i>		or <i>lval atom atom</i>
		xor <i>lval atom atom</i>		not <i>lval atom</i>
		cast <i>typ lval atom</i>		vpc <i>typ lval atom</i>
		assert <i>pred</i>		assume <i>pred</i>
		cut <i>pred_clause</i>		ecut <i>epred_clause</i>
		rcut <i>rpred_clause</i>		ghost <i>typed_var⁺ : pred</i>
		call <i>id (atom*)</i>		inline <i>id (atom*)</i>
		inlinespec <i>id (atom*)</i>		nop

Instructions *add*, *adds*, *adc*, *adcs*, *sub*, *subc*, *subb*, *sbc*, *sbc*, *sbb*, *sbbs*, *mul*, *muls*, *mull*, *mulj*, *split*, and *spl* also have specific unsigned and signed versions with prefix “u” or “s”. For example, *uadd* and *sadd* are respectively unsigned and signed versions of *add*.

Sometimes a predicate has to be proved with facts that have been cut off.

CRYPTOLINE offers the specification of hints required to prove a predicate.

$pred_clause$	$::=$	$true$		$epred_clause \ \&\& \ rpred_clause$
$epred_clause$	$::=$	$epred$		$epred \ \mathbf{prove \ with} \ [prove_with]^+$
		$epred_clause^+$		
$rpred_clause$	$::=$	$rpred$		$rpred \ \mathbf{prove \ with} \ [prove_with]^+$
		$rpred_clause^+$		
cas	$::=$	singular		sage
		magma		mathematica
		macaulay2		maple
$prove_with$	$::=$	precondition		all cuts
		all assumes		all ghosts
		cuts $[\mathbb{N}^+]$		algebra solver cas
		algebra solver $smt : path$		range solver $path$

Note that the indices of $ecut$ and $rcut$ are numbered separately (starting from 0). When verifying algebraic properties, $rcut$ instructions are ignored. When verifying range properties, $ecut$ instructions are ignored. For example, consider the following program.

```

mov x 15@uint16;
ecut x = 15;
mov y 3@uint16;
cut y = 3 && and [x = 15@16, y = 3@16];
add z x y;
rcut z = 18@16;

```

If we want to prove $e \text{ prove_with } [cuts[1]] \ \&\& \ r \text{ prove_with } [cuts[1]]$, then $y = 3$ will be assumed when proving the algebraic property e while $z = 18@16$ will be assumed when proving the range property r .

A *procedure* is a parameterized program together with its specification (precondition and postcondition).

$$proc ::= \mathbf{proc} \ id \ (\ formal \) = \{ \ pre \} \ prog \ \{ \ post \}$$

The *formal parameters* of a procedure may be separated by a semicolon into *inout* and *out* variables.

$$formals ::= typed_var^* \mid typed_var^* ; typed_var^*$$

Variables before the semicolon are inout variables while variables after the semicolon are out variables. Formal parameters without a semicolon are all inout variables. The difference between inout and out variables is that when calling a procedure, actual parameters of the inout formal variables must be defined but this is not required for the actual parameters of the out formal variables. However, this does not mean that an out variable can be read before initialized. Every variable must be initialized before reading its value. A *precondition* is a predicate.

$$pre ::= pred$$

A *postcondition* is a predicate clause.

$$post ::= pred_clause$$

A *statement* is a declaration of a procedure or a named integer.

$$stmt ::= proc \mid \textbf{const } id = const$$

A *program* is a sequence of semicolon separated statements. The entry point of the program is the *main* procedure. Other procedures called in main are inlined.

$$prog ::= stmt;^+$$